SMART CHEF & PANTRY AI

Flutter Clean Architecture & Proje Yapısı

Versiyon: 1.0
Frontend: Flutter / Dart
Architecture: Feature-Based Clean Architecture
State Management: Riverpod
Networking: Dio
Routing: go_router
Serialization: json_serializable

---

1. Amaç

Smart Chef & Pantry AI Flutter uygulaması, özellik bazlı Clean Architecture yaklaşımıyla geliştirilecektir.

Ana hedefler:

- UI ile business logic'i ayırmak
- API işlemlerini ekranlardan bağımsız hale getirmek
- Test edilebilir bir yapı oluşturmak
- Recipe matching gibi önemli logic'i UI içerisine taşımamak
- Yeni özellik eklerken mevcut kodu bozmamak
- Flutter uygulamasını Flask API'den bağımsız bir şekilde organize etmek

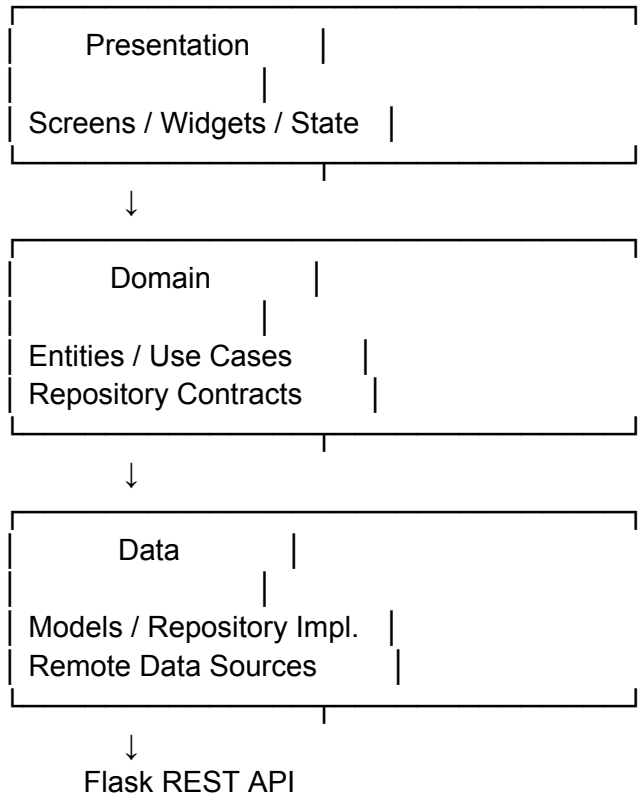
Temel mimari:

Presentation
↓
Domain
↓
Data
↓
REST API

---

2. Genel Flutter Mimarisi

Uygulamanın genel akışı:



Burada Dependency yönü önemlidir.

Presentation katmanı doğrudan Dio kullanmayacaktır.

Örneğin yanlış yapı:

RecipeScreen
↓
Dio
↓
API

Doğru yapı:

RecipeScreen
↓
RecipeProvider
↓
GetRecipeRecommendationsUseCase
↓
RecipeRepository
↓
RecipeRepositoryImpl

↓
RecipeRemoteDataSource
↓
Dio
↓
Flask API

---

3. Feature-Based Klasör Yapısı

Önerilen ana yapı:

```
lib/
│
├── core/
│   ├── constants/
│   ├── errors/
│   ├── network/
│   ├── routes/
│   ├── theme/
│   └── utils/
│
├── features/
│   │
│   ├── auth/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   │
│   ├── ingredients/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   │
│   ├── recipes/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   │
│   ├── favorites/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   │
│   ├── cart/
```

```
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   │
│   └── profile/
│       ├── data/
│       ├── domain/
│       └── presentation/
│
├── app.dart
└── main.dart
```

---

4. Core Katmanı

`core` bütün feature'ların ortak kullanacağı altyapıyı içerir.

```
core/
├── constants/
├── errors/
├── network/
├── routes/
├── theme/
└── utils/
```

Feature'a özel kod burada bulunmamalıdır.

Örneğin:

RecipeCard

core'a konulmamalıdır.

Çünkü RecipeCard sadece recipes feature'ına aittir.

---

5. Core Constants

```
core/constants/
├── api_constants.dart
├── app_constants.dart
└── storage_keys.dart
```

api_constants.dart

Örneğin:

baseUrl
auth endpoints
recipe endpoints
cart endpoints
favorite endpoints

gibi sabitler burada tutulacaktır.

Örnek:

```
class ApiConstants {
  static const String baseUrl =
      'http://10.0.2.2:5000/api';

  static const String ingredients =
      '/ingredients';

  static const String recommendRecipes =
      '/recipes/recommend';

  static const String favorites =
      '/favorites';

  static const String cart =
      '/cart';
}
```

Production ortamında base URL kod içerisine sabit yazılmamalı, environment/configuration üzerinden yönetilmelidir.

---

6. Core Network

```
core/network/
├── dio_client.dart
├── auth_interceptor.dart
└── network_info.dart
```

DioClient

Dio instance'ının merkezi yönetimini sağlar.

Sorumlulukları:

- Base URL
- timeout
- headers
- interceptors
- response handling

---

7. Authentication Interceptor

Authenticated request'lerde:

Authorization: Bearer <JWT>

header'ının otomatik eklenmesi planlanmaktadır.

Akış:

Flutter API Request
↓
AuthInterceptor
↓
JWT eklenir
↓
Dio
↓
Flask

Böylece her repository içerisinde manuel olarak:

```
headers: {
  'Authorization': 'Bearer $token'
}
```

yazılmasına gerek kalmaz.

---

8. Core Errors

core/errors/
├── api_exception.dart
├── failure.dart
└── exception_handler.dart

Amaç backend hatalarını Flutter'ın anlayabileceği ortak bir yapıya dönüştürmektir.

Örneğin backend:

```
{
  "success": false,
  "error": {
    "code": "RECIPE_NOT_FOUND",
    "message": "Recipe not found."
  }
}
```

döndürürse Flutter tarafında:

RecipeNotFoundFailure

veya ortak:

Failure

yapısı kullanılabilir.

---

9. Core Routes

core/routes/
└── app_router.dart

go_router ile uygulamanın route'ları burada tanımlanacaktır.

Örneğin:

/
├── home
├── ingredients

```
├── recipes
│   └── :id
├── favorites
├── cart
├── profile
└── login
```

Protected route kontrolü de burada veya authentication state üzerinden yapılacaktır.

---

10. Feature Yapısı

Her ana feature üç katmana ayrılacaktır:

```
feature/
├── data/
├── domain/
└── presentation/
```

Örneğin:

```
recipes/
├── data/
├── domain/
└── presentation/
```

---

11. Data Layer

Data layer dış sistemlerle iletişimden sorumludur.

```
recipes/data/
├── datasources/
├── models/
└── repositories/
```

Data Layer Sorumlulukları

- API çağrıları
- JSON parse
- DTO/model dönüşümleri

- Repository implementation
- Remote data source

---

12. Recipe Data Source

recipes/data/datasources/
└── recipe_remote_data_source.dart

Sorumluluğu:

Flask API
  ↕
RecipeRemoteDataSource

Örneğin:

```
abstract class RecipeRemoteDataSource {
  Future<RecipeModel> getRecipe(String id);

  Future<List<RecipeRecommendationItem>>
      recommendRecipes(List<String> ingredientIds);
}
```

Implementation:

RecipeRemoteDataSourceImpl

Dio kullanacaktır.

---

13. Data Models

recipes/data/models/
├── recipe_model.dart
├── recipe_ingredient_model.dart
├── recipe_recommendation_item_model.dart
└── recipe_recommendation_request_model.dart

Bunlar JSON ile iletişim için kullanılacaktır.

Örneğin:

```
factory RecipeModel.fromJson(
  Map<String, dynamic> json
)
```

ve:

```
Map<String, dynamic> toJson()
```

metotları bulunabilir.

`json_serializable` kullanılması planlanmaktadır.

---

14. Repository Implementation

```
recipes/data/repositories/
└── recipe_repository_impl.dart
```

Repository implementation:

```
Domain Repository
      ↑
      │ implements
      │
RecipeRepositoryImpl
      │
      ↓
RemoteDataSource
```

Örneğin:

```
class RecipeRepositoryImpl
   implements RecipeRepository {
}
```

Bu sınıf API'den gelen modelleri domain katmanının kullanacağı yapılara dönüştürebilir.

---

15. Domain Layer

Domain katmanı uygulamanın business logic sözleşmesini içerir.

recipes/domain/
├── entities/
├── repositories/
└── usecases/

---

16. Domain Entity

Entity API modelinden bağımsızdır.

Örneğin:

RecipeModel

API/data katmanına aittir.

Recipe

domain katmanına aittir.

Bu ayrım sayesinde API response'u değişse bile uygulamanın business logic'i doğrudan etkilenmez.

Örneğin:

RecipeModel
↓
Recipe Entity
↓
UseCase

---

17. Recipe Repository Contract

recipes/domain/repositories/
└── recipe_repository.dart

Örneğin:

```
abstract class RecipeRepository {
  Future<Recipe> getRecipe(String id);

  Future<List<RecipeRecommendation>>
      recommendRecipes(
        List<String> ingredientIds,
      );
}
```

Domain katmanı Dio veya Flask'ın varlığını bilmez.

Bu çok önemlidir.

Domain sadece:

"Recipe almak istiyorum."

der.

Nasıl alındığını Data Layer bilir.

---

18. Use Case Katmanı

```
recipes/domain/usecases/
├── get_recipe_detail.dart
└── recommend_recipes.dart
```

Her önemli kullanıcı işlemi bir use case olarak modellenebilir.

Örneğin:

```
RecommendRecipes
GetRecipeDetail
AddFavorite
RemoveFavorite
AddCartItem
UpdateCartItem
```

---

19. RecommendRecipes Use Case

Akış:

RecipeScreen
↓
RecommendRecipes
↓
RecipeRepository
↓
API

Örnek:

```
class RecommendRecipes {
  final RecipeRepository repository;

  RecommendRecipes(this.repository);

  Future<List<RecipeRecommendation>> call(
    List<String> ingredientIds,
  ) {
    return repository.recommendRecipes(ingredientIds);
  }
}
```

Use case'in amacı API çağrısını tekrar etmek değildir.

Use case, uygulamanın kullanıcı aksiyonunu temsil eder.

---

20. Presentation Layer

```
recipes/presentation/
├── pages/
├── widgets/
└── providers/
```

pages

Tam ekranlar:

recipe_results_page.dart
recipe_detail_page.dart

widgets

Tekrar kullanılabilir UI parçaları:

recipe_card.dart
ingredient_chip.dart
match_percentage_badge.dart
missing_ingredient_item.dart

providers

State management:

recipe_provider.dart
recipe_detail_provider.dart

---

21. Riverpod State Management

Riverpod uygulamadaki state'leri yönetmek için kullanılacaktır.

Örneğin:

Ingredient Selection State
Recipe Results State
Recipe Detail State
Favorite State
Cart State
Authentication State

---

22. Recipe Results State

Recipe recommendation ekranında üç temel durum olacaktır:

Initial
Loading
Success
Error

Örnek:

RecipeResultsState
├── initial
├── loading
├── success
└── error

Success:

recipes: List<RecipeRecommendation>

Error:

message

---

23. Ingredient Selection State

Kullanıcının seçtiği ingredient ID'leri Flutter tarafında tutulacaktır.

Örneğin:

```
selectedIngredients = [
  ingredient-1,
  ingredient-4,
  ingredient-8
]
```

Kullanıcı:

[Tarifleri Göster]

butonuna bastığında:

selectedIngredients
↓
RecommendRecipes
↓
POST /api/recipes/recommend

---

24. Authentication State

Authentication state:

AuthState
├── unauthenticated
├── authenticating
├── authenticated
└── error

Authenticated durumda:

User
+
JWT

bilgileri bulunacaktır.

JWT mümkün olduğunca güvenli storage üzerinde saklanacaktır.

---

25. Login Sonrası Route Davranışı

Kullanıcı guest olarak recipe detail ekranındayken:

Recipe Detail
↓
Favorite
↓
Login
↓
Google Authentication
↓
Recipe Detail

şeklinde geri dönmelidir.

Aynı durum cart için de geçerlidir.

Bu nedenle login ekranına giderken önceki route/context bilgisi gerektiğinde korunacaktır.

---

26. Cart Architecture

```
cart/
├── data/
│   ├── datasources/
│   │   └── cart_remote_data_source.dart
│   ├── models/
│   │   ├── cart_item_model.dart
│   │   └── cart_response_model.dart
│   └── repositories/
│       └── cart_repository_impl.dart
│
├── domain/
│   ├── entities/
│   │   └── cart_item.dart
│   ├── repositories/
│   │   └── cart_repository.dart
│   └── usecases/
│       ├── get_cart.dart
│       ├── add_cart_item.dart
│       ├── update_cart_item.dart
│       └── remove_cart_item.dart
│
└── presentation/
    ├── pages/
    │   └── cart_page.dart
    ├── widgets/
    │   └── cart_item_tile.dart
    └── providers/
        └── cart_provider.dart
```

---

27. Favorites Architecture

```
favorites/
├── data/
├── domain/
└── presentation/
```

Use case'ler:

GetFavorites
AddFavorite
RemoveFavorite

---

28. Ingredients Architecture

```
ingredients/
├── data/
│   ├── datasources/
│   ├── models/
│   └── repositories/
│
├── domain/
│   ├── entities/
│   ├── repositories/
│   └── usecases/
│
└── presentation/
    ├── pages/
    ├── widgets/
    └── providers/
```

Önemli use case'ler:

GetIngredients
SearchIngredients

Ingredient selection state presentation tarafında tutulabilir.

---

29. Auth Architecture

```
auth/
├── data/
│   ├── datasources/
│   │   └── auth_remote_data_source.dart
│   ├── models/
│   │   ├── user_model.dart
│   │   ├── social_login_request.dart
│   │   └── social_login_response.dart
│   └── repositories/
│       └── auth_repository_impl.dart
│
├── domain/
```

```
│   ├── entities/
│   │   └── user.dart
│   ├── repositories/
│   │   └── auth_repository.dart
│   └── usecases/
│       ├── social_login.dart
│       ├── get_current_user.dart
│       └── logout.dart
│
└── presentation/
    ├── pages/
    │   └── login_page.dart
    ├── widgets/
    └── providers/
        └── auth_provider.dart
```

---

30. Profile Architecture

MVP'de profile ekranı basit tutulacaktır.

```
profile/
├── data/
├── domain/
└── presentation/
```

İlk sürümde:

- Kullanıcı adı
- E-posta
- Profil fotoğrafı
- Logout

yeterlidir.

Ayarlar, bildirimler, kişiselleştirme gibi özellikler daha sonra eklenebilir.

---

31. App ve Main

Root seviyesinde:

```
lib/
```

```
├── app.dart
└── main.dart
```

main.dart

Uygulamanın başlangıç noktasıdır.

Örneğin sorumlulukları:

Flutter initialization
↓
Dependency initialization
↓
ProviderScope
↓
App

app.dart

Uygulamanın:

- Theme
- Router
- Global configuration

gibi yapılarını bir araya getirir.

---

32. Önerilen Tam Klasör Yapısı

```
lib/
│
├── core/
│   ├── constants/
│   │   ├── api_constants.dart
│   │   ├── app_constants.dart
│   │   └── storage_keys.dart
│   │
│   ├── errors/
│   │   ├── api_exception.dart
│   │   ├── failure.dart
│   │   └── exception_handler.dart
│   │
│   ├── network/
```

```
│   │   ├── dio_client.dart
│   │   └── auth_interceptor.dart
│   │
│   ├── routes/
│   │   └── app_router.dart
│   │
│   ├── theme/
│   │   ├── app_theme.dart
│   │   └── app_colors.dart
│   │
│   └── utils/
│       ├── validators.dart
│       └── extensions.dart
│
├── features/
│   │
│   ├── auth/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   │
│   ├── ingredients/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   │
│   ├── recipes/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   │
│   ├── favorites/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   │
│   ├── cart/
│   │   ├── data/
│   │   ├── domain/
│   │   └── presentation/
│   │
│   └── profile/
│       ├── data/
│       ├── domain/
│       └── presentation/
│
├── app.dart
└── main.dart
```

---

33. Dependency Injection

Provider'lar üzerinden dependency injection uygulanacaktır.

Örneğin:

Dio
↓
RemoteDataSource
↓
Repository
↓
UseCase
↓
Provider
↓
Screen

Bir sınıfın kendi dependency'sini oluşturması yerine dependency dışarıdan sağlanacaktır.

Yanlış:

```
class RecipeRepository {
  final dio = Dio();
}
```

Doğru yaklaşım:

Dio Provider
↓
DataSource Provider
↓
Repository Provider
↓
UseCase Provider

Bu yaklaşım testlerde mock dependency kullanmayı kolaylaştırır.

---

34. UI ile Business Logic Ayrımı

Recipe ekranında şu kodların bulunması istenmez:

HTTP request
JSON parsing
SQL logic
match percentage calculation
authentication token management

Ekranın görevi:

UI oluştur
↓
User action al
↓
Provider'ı çağır
↓
State'i göster

Örneğin:

[Tarifleri Göster]
↓
provider.recommendRecipes()
↓
loading
↓
success/error

---

35. Recipe Matching Nerede Olacak?

Matching algorithm Flutter'da olmayacaktır.

Flutter
↓
POST /api/recipes/recommend
↓
Flask
↓
Recipe Matching Service
↓

PostgreSQL

Bu karar özellikle korunmalıdır.

Çünkü algoritma ileride:

Rule Based
↓
Weighted Matching
↓
Content Based Recommendation
↓
ML Recommendation

şeklinde geliştirilebilir.

Flutter'ın bu değişikliklerden haberdar olması gerekmez.

---

36. State Management Sınırları

Riverpod her şeyi yönetmek için kullanılmayacaktır.

Örneğin:

UI state

Loading
Selected ingredient
Current filter

Riverpod tarafından yönetilebilir.

Server state

Recipes
Favorites
Cart
User

API ile senkronize edilen state olarak yönetilecektir.

Business logic

Recipe matching
Cart merge
Authentication validation

bunlar provider'ın içine gömülmeyecek, ilgili domain/service katmanlarında tutulacaktır.

---

37. Navigation Yapısı

Temel route yapısı:

/
└── Home

/search
└── Ingredient Selection

/recipes
└── Recipe Results

/recipes/:id
└── Recipe Detail

/favorites
└── Favorites

/cart
└── Cart

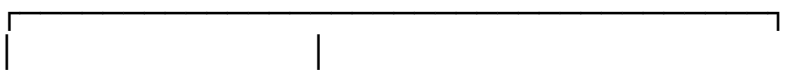
/profile
└── Profile

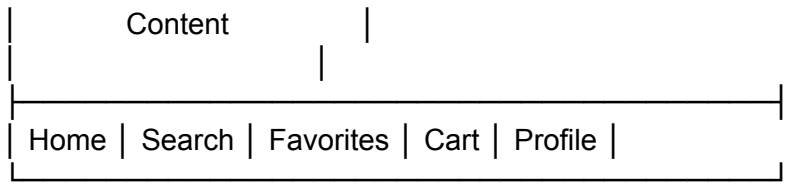
/login
└── Social Login

---

38. Bottom Navigation

Ana uygulama shell'i:





Guest kullanıcı:

Favorites
Cart
Profile

gibi protected bölümlere girdiğinde authentication kontrolü yapılacaktır.

---

39. API Loading ve Error UX

API işlemlerinde kullanıcıya üç temel durum gösterilecektir:

Loading
Success
Error

Örneğin recipe recommendation:

Loading:
"Tarifler hazırlanıyor..."

Success:
Recipe List

Error:
"Tarifler yüklenirken bir sorun oluştu."
[Tekrar Dene]

Backend'den gelen teknik hata mesajı doğrudan kullanıcıya gösterilmemelidir.

Örneğin:

DATABASE_ERROR
SQLAlchemy exception

gibi teknik bilgiler UI'a verilmemelidir.

---

40. Test Edilebilirlik

Mimari aşağıdaki testleri mümkün hale getirmelidir:

Unit Tests
  ├── UseCases
  ├── Validators
  └── State logic

Repository Tests
  └── Mock DataSource

Widget Tests
  └── UI components

Integration Tests
  └── API + Flutter flow

Özellikle:

RecommendRecipes
AddFavorite
AddCartItem
UpdateCartItem

use case'leri test edilebilir olmalıdır.

---

41. Mimari Karar Özeti

Smart Chef & Pantry AI Flutter tarafında:

Feature-Based Clean Architecture
+
Riverpod
+
Dio
+
go_router

+
json_serializable

kullanılacaktır.

Katmanlar:

Presentation
↓
Domain
↓
Data
↓
REST API

şeklinde organize edilecektir.

En önemli kurallar:

1. UI doğrudan API çağırmaz.
2. UI business logic içermez.
3. Domain Dio'yu bilmez.
4. Database modeli Flutter'a doğrudan taşınmaz.
5. Recipe matching backend'de yapılır.
6. Authentication state merkezi olarak yönetilir.
7. Protected route'lar authentication state ile kontrol edilir.
8. API hataları ortak bir modele dönüştürülür.
9. Feature'lar birbirine gereksiz şekilde bağımlı hale getirilmez.
10. MVP'de gereksiz abstraction oluşturulmaz.

---

42. Sonraki Aşama

Flutter mimarisi belirlendikten sonra backend tarafının da aynı disiplinle tasarlanması gerekmektedir.

Bir sonraki doküman:

Doküman 08 — Flask Backend Architecture

Bu dokümanda:

Flask App
↓
Routes
↓
Schemas
↓
Services
↓
Repositories
↓
SQLAlchemy Models
↓
PostgreSQL

yapısını oluşturacağız.

Özellikle `RecipeMatchingService`, `AuthService`, `CartService` ve `FavoriteService` sınıflarının görevlerini de kesin olarak ayıracağız.